Oral Exam: Machine Learning (2ST129)

Måns Magnusson, Department of Statistics, Uppsala University

Contents

Oral Exam Instructions **2**

Oral Exam Questions **4**

Assignment 1: Optimisation and Regularisation 4
Assignment 2: Trees and Ensembles 6
Assignment 3: Feed-forward Neural Networks 7
Assignment 4: Convolutional Neural Networks 9
Assignment 5: Sequence Models and Attention 10
Assignment 6: Clustering and Probabilistic PCA 12
Assignment 7: Variational Autoencoders and Topic Models 14
Assignment 8: Bandits and Markov Decision Processes 15

Oral Exam Instructions

Preparation

- Review the questions for the assignments covered by the exam, together with the corresponding reading and submitted work.
- Practise explaining your answers aloud, preferably by asking each other questions in small groups.
- You should be able to explain every part of your submitted assignments, including functions you implemented. For a function walkthrough, explain its purpose, inputs and outputs (including dimensions), and main steps. Connect the code to the method and an example from your report.
- You are not expected to memorise formulas. Starting formulas for derivations are provided in the questions. Explain the steps, assumptions, and interpretation rather than memorising a finished derivation. If a necessary formula is missing, contact the examiner in advance so it can be added.

Sessions and Eligibility

- Two hours are reserved for each examination session; examinations normally use 1–2 hours, depending on the number of students. Dates, times, and rooms are announced in Studium and/or TimeEdit.
- You may be examined only on written assignments that you have already submitted in Studium. You do not need to wait for the written assignment to be graded.
- You may be examined on *a maximum of three assignments per examination session*, including assignments taken as re-examinations. There may be multiple examination sessions in the same week.

Order and Attendance

- The order of students will be randomised and shared before the exam. Check the order and be present and prepared when it is your turn.
- If you miss your turn, you must take the exam at a later available examination opportunity. Missing an opportunity does not, by itself, remove your eligibility for VG.

During the Exam

- You will receive one question for each assignment covered by the exam, drawn at random from the eligible questions in the relevant assignment section below.
- Questions marked *** (VG only)** apply only if you are seeking VG for that assignment and submitted the corresponding VG task. Otherwise, they are excluded from the random draw, including replacement questions.
- Answer orally. You may also use a whiteboard and pen to explain your reasoning.
- The examiner may ask follow-up questions to clarify your answer or assess your understanding.
- In addition to the predefined questions below, the examiner may ask questions about any part of your own submitted assignments, including your code, results, and reasoning. This may include walking through a selected function or connected part of your implementation, using your submitted code as a reference. You do not need to memorise code. Questions about optional VG implementations apply only if you are seeking VG for that assignment and submitted the corresponding VG work.

Time Needed

- The time needed can vary between students and between questions.
- Some students may demonstrate sufficient understanding of a question in about 20 seconds. Others may need more time and follow-up questions.
- This is not a speed test. We assess your understanding, not how quickly you answer. A longer discussion or a follow-up question does not, by itself, mean that you have failed.

Passing Each Assignment

- Each assignment is assessed separately.
- If your first answer is satisfactory, you pass the oral exam for that assignment.
- If your first answer is not satisfactory, you receive two new questions drawn at random from the same assignment section, excluding the question already attempted. You must answer both satisfactorily to pass. If either answer is not satisfactory, you fail that attempt for the assignment.
- These two new questions are separate from follow-up questions used to clarify an answer. An unsatisfactory first answer is not a failed attempt if you then answer both new questions satisfactorily.

VG Eligibility

- Failing an oral-exam attempt for an assignment means that you cannot receive VG for that assignment, even if you pass a later attempt.
- Missing an examination opportunity does not remove your eligibility for VG. You must still pass the oral exam for the assignment at a later opportunity and meet the other VG requirements.

Exams and Re-examinations

- There will be two additional re-examination sessions after the end of the course.
- Later sessions provide opportunities to retake previously failed or missed oral exams. Re-examinations and new assignment examinations may take place during the same session, within the limit of three assignments per examination session.
- Dates for the two additional re-examination sessions will be announced in Studium and/or TimeEdit.
- You must have passed the oral exams for all assignments by the end of the second of the two additional re-examination sessions after the course. If you have not, you fail the course and must retake it in full the next time it is offered. This applies whether earlier opportunities were failed or missed.

Oral Exam Questions

Assignment 1: Optimisation and Regularisation

1. **Explain the difference between gradient descent, stochastic gradient descent, and mini-batch gradient descent. When might you prefer one over another?**

2. **For binary logistic regression, let $y_i \in \{0, 1\}$ and $p_i = \sigma(x_i^T \theta)$, where $\sigma(z) = 1/(1 + e^{-z})$. The negative log-likelihood contribution from observation i is**

$$\ell_i(\theta) = -y_i \log p_i - (1 - y_i) \log(1 - p_i).$$

Using $\sigma'(z) = \sigma(z)[1 - \sigma(z)]$, derive the gradient of $\ell_i(\theta)$ with respect to θ. Explain how the contributions combine for the whole dataset and interpret the term $p_i - y_i$.

3. **What does the learning rate (η) control in gradient descent, and what happens if it is too large or too small?**

4. **Give at least two examples of differences between *traditional regression methods* and the *pure prediction algorithms* according to Efron (2020).**

5. **In many machine learning applications, we use Stochastic Gradient Descent, even though other optimization algorithms that use second-order derivatives are better. Why is this the case?**

6. **Describe the concept of model complexity/capacity.**

7. **Explain the main difference between Ridge regression and LASSO.**

8. **What is the difference between training error and generalization error? Why is it important to distinguish between them?**

9. **Explain the bias–variance trade-off in supervised learning.**

10. **What is the purpose of cross-validation, and how does k-fold cross-validation work?**

11. **Describe how regularization affects the objective function in regression models.**

12. **Explain the probabilistic interpretation of L2 (ridge) regularization.**

13. **What are the main differences between linear regression and logistic regression in terms of model output and objective function?**

14. **What is the role of the objective (or loss) function in supervised learning?**

15. **How does stochastic gradient descent (SGD) differ in convergence behavior compared to full-batch gradient descent?**

16. **How does regularization help prevent overfitting in machine learning models?**

17. **What is the main difference in the effects of L1 and L2 penalties on model coefficients?**

18. **When using cross-validation to choose a regularization parameter (e.g. λ), what does the optimal λ represent?**

19. **Derive a ridge gradient. For $J(\theta) = n^{-1}\|y - X\theta\|^2 + \lambda\|\theta\|^2$, derive the gradient and write one gradient descent update. All coefficients in this expression are penalised.**

20. **Explain the no-free-lunch theorem for supervised learning in your own words. What does it imply about choosing a learning algorithm?**

21. **Ruder (2016) presents mini-batch gradient descent as a compromise between batch and stochastic gradient descent. Explain this compromise using your implementations with batch sizes n, 10, and 1.**

22. **Walk us through your `nll_grad()` implementation from Assignment 1. Explain its inputs and output, how the code computes the gradient of the mean negative log-likelihood, and the result of one example from your report.**

23. **Walk us through your `gradient_descent()` implementation from Assignment 1. Explain its inputs and returned history, then trace one batch update. How does the same code handle full-batch, stochastic, and mini-batch gradient descent? Use an example from your report.**

24. **Walk us through your `glmnet_cv()` implementation from Assignment 1, or your equivalent cross-validation function. Explain its inputs and output, and follow one fold through fitting, prediction, and error calculation. How do you use the result to choose λ? Use an example from your report.**

Assignment 2: Trees and Ensembles

1. **Explain how a decision tree decides where to split the data. What criterion is used and what is its goal?**

2. **What is the main idea behind bagging (bootstrap aggregating), and why does it improve model performance?**

3. **Describe how a random forest differs from bagging. What additional randomness does it introduce, and why?**

4. **What is the fundamental idea behind boosting, and how does it differ conceptually from bagging?**

5. **In ensemble methods like bagging and boosting, what happens to bias and variance as more models (e.g. trees) are added?**

6. **What is "overfitting" in a decision tree, and how can it be prevented?**

7. **Why is randomness crucial in ensemble methods like random forests?**

8. **What does the parameter m (number of features sampled at each split) control in a random forest?**

9. **What role does the learning rate play in boosting algorithms?**

10. **Walk us through your `tree_split()` implementation from Assignment 2. Explain its inputs and output, how it chooses a split, and how it enforces the minimum leaf size. Use an example from your report.**

11. **Walk us through your `grow_tree()` implementation from Assignment 2. Explain how it uses `tree_split()`, stores the tree, and decides when to stop splitting. Use an example from your report.**

12. **Walk us through your `predict_with_tree()` implementation from Assignment 2. Explain its inputs and output, and follow one new observation through your stored tree to its prediction. Use an example from your report.**

Assignment 3: Feed-forward Neural Networks

1. **Describe the output layer in a (feed-forward) neural network. What does it do?**

2. **Describe the input layer in a (feed-forward) neural network. What does it do?**

3. **What is the benefit of using (negative) log-likelihood as loss functions?**

4. **Suppose a neural network is trained using gradient descent without momentum, with objective**

$$J(w) = L_{\text{data}}(w) + \lambda \sum_j w_j^2, \qquad \lambda > 0.$$

   Derive one parameter update with learning rate η. Explain why the penalty pulls weights towards zero.

5. **How can L_2 regularisation of the weights help reduce overfitting in a neural network?**

6. **What can happen if the L_2 regularisation strength λ is too large when training a neural network?**

7. **What is the role of activation functions in a neural network, and why are ReLU (or its variants) often preferred?**

8. **What is backpropagation, and how does it enable a neural network to learn?**

9. **Explain how batch normalization works and why it improves training stability and generalization.**

10. **Describe how early stopping can be used as a regularization technique in neural networks.**

11. **What is transfer learning, and when is it especially useful?**

12. **What does dropout do during training, and how does it help prevent overfitting?**

13. **What are the main components of a feed-forward neural network architecture?**

14. **Why do we typically use softmax activation in the output layer for multi-class classification?**

15. **What is the purpose of using validation accuracy and loss curves during training?**

16. **How does transfer learning benefit from freezing early layers in a pre-trained model?**

17. **What is fine-tuning in transfer learning, and how does it differ from feature extraction?**

18. **What loss functions are typically used for regression and classification tasks in neural networks, and why?**

19. **Why does using too many hidden layers or neurons not always improve performance?**

20. **What are frozen layers in transfer learning, and why might we unfreeze some of them later?**

21. **What evaluation metrics besides accuracy can be useful for assessing a neural network's performance, and when?**

22. **What advantages can a deep neural network have over a shallow one? Explain the role of composing and reusing features, and why greater depth does not guarantee better performance.**

23. **Explain the relationship between bagging and dropout regularisation. In what sense can dropout be viewed as training an ensemble, and how does it differ from ordinary bagging?**

24. **Walk us through your Keras network code from Assignment 3. Explain the layers, input and output dimensions, activations, and loss function. What does training update, and how do you use validation and test data? Use a model from your report.**

25. *** (VG only) Walk us through your `mini_net()` implementation from the Assignment 3 VG task. Explain the inputs, weights, biases, hidden activation, and output dimensions. Follow one observation through the code using an example from your report.**

26. *** (VG only) Walk us through your `mini_net_loss()` implementation from the Assignment 3 VG task. Explain how it uses `mini_net()`, compares predictions with observations, and returns a loss. Use an example from your report.**

Assignment 4: Convolutional Neural Networks

Starred questions apply only if you are seeking VG for Assignment 4 and submitted the specified option. Manual convolution and VGG16 transfer learning are alternative VG options.

1. **What advantages do convolutional layers offer compared to fully connected layers when processing image data?**

2. **Explain the purpose of pooling layers in CNNs and how max pooling works.**

3. **How does padding affect the output of a convolutional layer, and why might we use it?**

4. **Explain what a convolutional kernel (filter) is and how it is learned during training.**

5. **Why do deeper CNNs generally perform better than shallower ones, and what are the risks of increasing depth too much?**

6. **What is the difference between stride and pooling in CNNs?**

7. **What is transfer learning, and how can pretrained CNNs like VGG16 improve performance on small datasets?**

8. **When performing hyperparameter tuning, why is random search often preferred over grid search?**

9. **Describe data augmentation and how it can improve CNN generalization.**

10. **What does "feature map" mean in the context of CNNs, and what does it represent?**

11. **Work out the size of a convolutional layer. For one spatial dimension, use $o = \lfloor (n + 2p - k)/s \rfloor + 1$, where n is input size, p padding on each side, k kernel size, and s stride. Find the output shape and number of trainable parameters for a $28 \times 28 \times 1$ input, eight 3×3 filters, stride one, no padding, and one bias per filter. Explain why spatial size does not multiply the parameter count.**

12. **When would gathering more training data be a useful next step in an ML application? Explain how training and validation learning curves can help you decide.**

13. **Walk us through one CNN from your Assignment 4 report. Follow an image through its layers and explain the changing dimensions, activations, and final class probabilities.**

14. **Explain your hyperparameter-search code from Assignment 4. How did you sample settings, create fresh models, compare validation results, and select the model for final test evaluation?**

15. *** (VG only) For the manual convolution option, walk us through your `convolution()` implementation. Explain how one image window and kernel produce one output value, and how stride and padding determine the output dimensions.**

16. *** (VG only) For the manual convolution option, walk us through your `convolutional_layer()` implementation. Explain how it combines convolution, bias, and activation. Use a feature map from your report to explain their effects.**

17. *** (VG only) For the manual convolution option, walk us through your `maxpool_layer()` implementation. Follow one pooling window and explain which value is retained, how the window moves, and the resulting output dimensions.**

Assignment 5: Sequence Models and Attention

Starred questions apply only if you are seeking VG for Assignment 5 and submitted the specified option. Keras RNN/LSTM and BERT are alternative VG options.

1. **How do word embeddings differ from one-hot encoding, and why are embeddings useful in NLP?**

2. **Explain the role of the query, key, and value matrices in the transformer attention mechanism.**

3. **Why do transformers use positional encoding, and how does it work?**

4. **What is multi-head attention, and why is it beneficial compared to single-head attention?**

5. **What is BERT, and how does it differ from the original transformer model?**

6. **What does "self-attention" mean in the transformer architecture?**

7. **What are word embeddings like Word2Vec or GloVe missing that contextual embeddings like BERT provide?**

8. **What is the main architectural difference between the transformer encoder and decoder?**

9. **How does fine-tuning differ from pretraining in transformer models?**

10. **What is tokenization in NLP models, and why do models like BERT use WordPiece tokenization?**

11. **How does GPT differ architecturally and conceptually from BERT?**

12. **What is the difference between a recurrent neural network and an ordinary feed-forward neural network? Explain how an RNN represents information from earlier elements of a sequence.**

13. **In Assignment 5, you ran attention on the first three words and then on the full sentence. Why can the output vector for the word "quick" change, even though its input embedding is unchanged? Explain using the attention mechanism described in Section 3.2 of Vaswani et al. (2017).**

14. **Walk us through your `qkv()` implementation from Assignment 5. Explain its inputs and how it turns token embeddings into query, key, and value matrices. Use an example from your report to explain the output dimensions.**

15. **Walk us through your `attention()` implementation from Assignment 5. Explain the scaled scores, row-wise softmax, and weighted combination of values. Follow one output row using an example from your report.**

16. **Walk us through your `multi_head_self_attention()` implementation from Assignment 5. Explain how it uses separate heads, combines their outputs, and applies the output projection. Use your code and an example from your report.**

17. *** (VG only) For the Keras RNN/LSTM option in Assignment 5, walk us through your models and training code. Explain the input representation, sequence processing, output, and how you compared your LSTM modifications. Use results from your report.**

18. *** (VG only) For the BERT option in Assignment 5, walk us through your experiment. Explain how text reaches the classifier, what you changed from the supplied baseline, and how you evaluated the change. Use results from your report.**

Assignment 6: Clustering and Probabilistic PCA

Starred questions apply only if you are seeking VG for Assignment 6 and submitted the EM section. They concern your submitted mixture-model implementations.

1. **What does k-means minimise? For $J = \sum_i \|x_i - \mu_{z_i}\|^2$, explain the roles of the cluster assignments z_i and centroids μ_k, and describe the two alternating steps.**

2. **Why should you try several initialisations of k-means, and how would you compare the results in your assignment?**

3. **Why can changing the units of one variable change k-means clusters? When might standardising variables be appropriate?**

4. **Explain the probabilistic PCA model $x = Wh + b + \epsilon$, with $h \sim N(0, I_q)$ and independent $\epsilon \sim N(0, \sigma^2 I_d)$. What do W, b, q, and σ^2 represent?**

5. **Derive the mean and covariance in probabilistic PCA. Using $x = Wh + b + \epsilon$, $E(h) = 0$, $\mathrm{Cov}(h) = I_q$, and independent noise with mean zero and covariance $\sigma^2 I_d$, derive $E(x)$ and $\mathrm{Cov}(x)$.**

6. **What is the effect of keeping fewer principal components in PCA? Explain the trade-off in terms of reconstruction and dimension reduction.**

7. **How would you simulate correlated observations from a probabilistic PCA model, and how would you check the simulated data?**

8. **What is unsupervised learning? Give two examples of tasks and explain how it differs from supervised learning.**

9. **What is the difference between the meaning of data augmentation in the EM algorithm and in training a CNN?**

10. **Describe the E-step and M-step of the EM algorithm. Use a Gaussian mixture model to illustrate what each step computes.**

11. **Compare a Gaussian mixture model with k-means clustering. How do their assignments, cluster shapes, and fitting objectives differ?**

12. **What is the difference between ordinary PCA and probabilistic PCA? Explain what the probabilistic formulation adds and how their principal subspaces are related.**

13. **Walk us through your `compute_cluster_means()` implementation from Assignment 6. Explain how cluster labels identify observations and how each mean vector is computed. Use an example from your report to explain the output dimensions.**

14. **Walk us through your `compute_cluster_encoding()` implementation from Assignment 6. Explain how it assigns an observation to a cluster and what it returns. Follow one observation using an example from your report.**

15. **Walk us through your `k_means()` implementation from Assignment 6. Explain initialisation, how the two update functions are combined, when the algorithm stops, and how your code handles an empty cluster. Use a run from your report.**

16. **Walk us through your `k_means_W()` implementation from Assignment 6. Explain how it computes the assignment's within-point scatter criterion. Trace one cluster's contribution using an example from your report.**

17. **Walk us through your `pPCA()` implementation from Assignment 6. Explain how it generates one observation from latent coordinates, loadings, a mean, and noise. Use your code and a simulated example from your report.**

18. *** (VG only) Walk us through your `r_uni_two_comp()` implementation from the Assignment 6 EM section. Explain how the component is chosen and how an observation is generated. Use an example from your report.**

19. *** (VG only) Walk us through your `d_uni_two_comp()` implementation from the Assignment 6 EM section. Explain how it combines the component densities and what it returns. Use an example from your report.**

20. *** (VG only) Walk us through your `e_uni_two_comp()` implementation from the Assignment 6 EM section. Explain how it computes responsibilities and interpret the result for one observation from your report.**

21. *** (VG only) Walk us through your `max_uni_two_comp()` implementation from the Assignment 6 EM section. Explain how responsibilities determine the updated means, standard deviations, and mixing weight. Use an example from your report.**

22. *** (VG only) Walk us through your `ll_uni_two_comp()` implementation from the Assignment 6 EM section. Explain how it computes the mixture log-likelihood and handles small density values. Use an example from your report.**

23. *** (VG only) Walk us through your `em_uni_two_comp()` implementation from the Assignment 6 EM section. Trace one complete iteration and explain the final parameters, log-likelihood output, and parameter history. Use a run from your report.**

Assignment 7: Variational Autoencoders and Topic Models

1. **What do the encoder and decoder do in a variational autoencoder? How does its latent representation differ from a deterministic autoencoder's?**

2. **Explain the VAE reparameterisation $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$, with $\epsilon \sim N(0, I)$. What does the sampling layer do, and why is this representation useful for training?**

3. **Interpret the two terms of the VAE objective**

$$\mathcal{L}(x) = E_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \| p(z)).$$

Which direction is optimised, and why is good reconstruction alone insufficient?

4. **Why is the ELBO a lower bound? Using**

$$\log p_\theta(x) = \mathcal{L}(x) + \mathrm{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x)),$$

explain the inequality and when it becomes an equality.

5. **How would you interpret a two-dimensional VAE plot of MNIST digits? Why need two independently trained models not use the same latent coordinates?**

6. **In a topic model, what do $\theta_{dk} = P(z = k \mid d)$ and $\phi_{kw} = P(w \mid z = k)$ represent? How do their rows sum, and why can one document contain several topics?**

7. **Why remove stopwords and very rare words before fitting a topic model? Is a stable log-likelihood trace sufficient to establish convergence or meaningful topics?**

8. **Using Figure 2.2 in Kingma and Welling (2019), describe the three steps in a VAE: the encoder, sampling a latent variable, and the decoder. For an MNIST image in your assignment, what goes into each step and what comes out? Explain how the three steps are connected.**

9. **In Figure 2, Griffiths and Steyvers (2004) show both a log-likelihood trace and how the estimated topics develop during sampling. Why are both useful? Connect their illustration to your own likelihood trace and your inspection of the resulting topics.**

10. **Blei (2012) introduces LDA through an article combining genetics, evolutionary biology, and data analysis. Explain how the model represents this combination. Then explain how the same idea applies to a paragraph containing several topics in your analysis of *Pride and Prejudice*.**

Assignment 8: Bandits and Markov Decision Processes

Starred questions apply only if you are seeking VG for Assignment 8 and submitted the corresponding VG bandit implementation.

1. **What is the exploration–exploitation trade-off in a multi-armed bandit? Explain why a greedy policy can remain with a suboptimal action.**

2. **Derive an incremental sample mean for a bandit action. Starting from $Q_n = n^{-1} \sum_{i=1}^{n} R_i$, derive Q_n in terms of Q_{n-1} and R_n. Why count observations of that action rather than all time steps?**

3. **How does an ϵ-greedy policy work? For K actions and a unique greedy action, what is the probability of selecting that action if exploration samples uniformly from all K actions?**

4. **Why can optimistic initial value estimates encourage exploration under a greedy policy? Does this guarantee continued exploration?**

5. **What information defines a Markov decision process? Explain state, action, transition and reward, and the Markov property using the recycling robot.**

6. **Compute and interpret a discounted return. Given $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$, rewards $(2, 0, 4)$, and $\gamma = 1/2$, find G_t. How is discounted return different from the average reward per time step reported in the simulation assignment?**

7. **What is reinforcement learning, and how does it differ from supervised learning? Explain the role of interaction and the goal of the learning agent.**

8. **When might a greedy bandit policy outperform an ϵ-greedy policy with $\epsilon = 0.1$? Explain both the benefit and the risk of avoiding exploration.**

9. **Walk us through your `stationary_bandit()` implementation from Assignment 8. Explain how the action determines the reward distribution, what the function returns, and why repeated calls can give different results. Use an example from your report.**

10. **Walk us through your `nonstationary_bandit()` implementation from Assignment 8. Explain how the action and time determine the reward distribution. Use an example from your report to distinguish the changing mean from a realised reward.**

11. **Walk us through your `greedy_algorithm()` implementation from Assignment 8. Trace one action selection, reward, count update, and value update. Explain the returned results using a run from your report.**

12. **Walk us through your `epsilon_algorithm()` implementation from Assignment 8. Explain how it chooses between exploration and exploitation, then trace one value update. Use a run from your report.**

13. **Walk us through your `nonstationary_algorithm()` implementation from Assignment 8. Explain action selection and the constant-step value update. How does this update differ from a sample average? Use a run from your report.**

14. **Walk us through your `always_search_policy()` implementation from Assignment 8. Follow one step from the current state to the next state and reward. Explain the state counts and mean reward using a run from your report.**

15. **Walk us through your `charge_when_low_policy()` implementation from Assignment 8. Explain how battery state determines the action and follow a recharge step. Use your reported state counts and mean reward.**

16. *** (VG only) Walk us through your `ucb_algorithm()` implementation from Assignment 8. Explain how it handles untried actions and combines value estimates with an exploration bonus. Trace one selection and update using a run from your report.**

17. *** (VG only) Walk us through your `gradient_bandit_algorithm()` implementation from Assignment 8. Explain preferences, action probabilities, the reward baseline, and one preference update. Use a run from your report.**